

173. Binary Search Tree Iterator

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: October 10, 2026

Problem Summary

Implement an iterator over a BST that supports `next()` — return the next smallest number, and `hasNext()` — return true if a next number exists. Both must run in $O(1)$ average time.

Approach

Run full in-order traversal in the constructor and store all values in an `ArrayList`. `next()` returns `list.get(index++)` and `hasNext()` checks `index < list.size()`. In-order of BST gives sorted ascending order — so `next()` naturally returns the next smallest.

Key Insights

- In-order of BST = sorted ascending — next smallest is just the next element in the list.
- `index++` in `next()` returns current value AND advances index in one step.
- `list` and `index` are instance variables — shared across all `next()/hasNext()` calls.
- Simple list approach trades $O(n)$ space for $O(1)$ time per call.

Observations

- Constructor does all the work — `next()` and `hasNext()` are trivially $O(1)$.
- Stack-based lazy approach uses $O(h)$ space — better for large trees with few iterator calls.
- `list.get(index++)` — post-increment returns current index value then increments.
- Works for any BST regardless of shape — in-order handles all cases.

Approaches

- **Brute Force / Simple List $O(n)$ space:** Run full in-order traversal in constructor, store all values in a list, maintain an index. `next()` returns `list.get(index++)`, `hasNext()` checks `index < list.size()`. $O(1)$ per call.
- **Optimal / Lazy Stack $O(h)$ space:** Use a Stack — push all left nodes of current path. `next()` pops top, then pushes left path of right child. $O(h)$ space, $O(h)$ amortized per call.

Algo

1. In constructor: call `inOrder(root)` to fill list with sorted values.
2. `inOrder`: if null return. Recurse left, add `root.val`, recurse right.
3. `next()`: return `list.get(index++)`.
4. `hasNext()`: return `index < list.size()`.

Dry Run

Call	index before	returns	index after
<code>next()</code>	0	3	1
<code>next()</code>	1	7	2
<code>hasNext()</code>	2	true	2
<code>next()</code>	2	9	3
<code>hasNext()</code>	3	true	3

BST in-order: [3, 7, 9, 15, 20]. list filled at construction, index walks through it.

Section

Time Complexity

$O(n)$ constructor, $O(1)$ `next()` and `hasNext()`.

Space Complexity

$O(n)$ — list stores all n node values.

Points to Remember

- BST iterator = in-order traversal + index pointer — direct application of BST sorted property.
- `index++` returns then increments — use in `next()` to return and advance in one line.
- Simple list vs stack: $O(n)/O(1)$ vs $O(h)/O(h)$ — mention both in interviews.
- `hasNext()` = `index < list.size()` — standard pattern for any list-backed iterator.
- Stack-based alternative: push all left nodes, `next()` pops and pushes right's left path — $O(h)$ space.

Solution Code — Java

```
class BSTIterator {
    List<Integer> list = new ArrayList<>();
    int index = 0;

    public BSTIterator(TreeNode root) {
        inOrder(root);
    }

    public void inOrder(TreeNode root) {
        if (root == null) return;
        inOrder(root.left);
        list.add(root.val);
        inOrder(root.right);
    }

    public int next() {
        return list.get(index++);
    }

    public boolean hasNext() {
        return index < list.size();
    }
}
```